

IFN657 Assignment 2: Vulnerability Discovery

- **Assessment type:** Group project (4 students per group; 3 by explicit approval only)
- **Weight:** 30% of total unit grade
- **Due date:** Friday 16 October (Week 12)
- **Project demonstration:** Week 13 tutorials
- **Deliverables:** Technical report (Group_XX.pdf compiled from as2_report_template.md) and project archive (Group_XX.zip containing all fuzzing artifacts, crash reproducers, code remediations, and Group_XX.cast)

1 Overview

Following the successful vulnerability audit and manual remediation of the station gateway controllers in Assignment 1, the Orbital Security Directorate has escalated its security evaluation to the core mission-critical flight computers of the *Sentinel-1 Space Station*. The communications hub and external sensor suites rely on three interconnected C firmware daemons that parse real-time environmental telemetry, deep-space scientific observations, and high-frequency ground telecommand packets:

1. `sentinel_telemetry.c`: The subsystem telemetry and environmental configuration parser. It ingests ground station directives, power distribution thresholds, and sensor calibration profiles.
2. `sentinel_payload.c`: The scientific observation and imaging frame parser. It processes multi-spectral observation matrices, radar scans, and variable-length scientific payloads.
3. `sentinel_network.c`: The inter-satellite telecommand protocol handler. It processes high-frequency radio-frequency (RF) packet frames, dispatching flight manoeuvres and emergency subsystem resets.

Due to the complexity of these high-throughput binary parsers, manual source code review alone cannot guarantee memory safety. As a specialist security engineering team, your objective is to design and execute a comprehensive automated vulnerability discovery campaign using AFL++, compiler-based sanitisers, and GDB. Your team will demonstrate the ability to perform effective fuzz testing, such as constructing seed corpora, configuring multi-core fuzzing pipelines, triaging crashes, performing root cause analyses, implementing code-level remediations, conducting regression verification, and constructing a functioning proof-of-concept exploit demonstrating actionable impact beyond a crash.

The submission must be your team's own original work. The use of generative artificial intelligence (GenAI) tools is prohibited during this assessment. This assignment is eligible for the 48-hour late submission period and assignment extensions.

2 Teamwork

You will complete this assessment in teams of 4 students. Teams of 3 are permitted only under exceptional circumstances with prior written approval from the unit coordinator. Team size does not alter assignment requirements or grading standards.

2.1 Workload Distribution

All members are expected to contribute equally and meaningfully to the practical fuzzing campaigns, vulnerability analyses, exploit development, and technical report authoring. *Any team member who does not contribute substantially will receive zero for this assessment.*

Hold regular team meetings to establish expectations, schedule shared testing sessions, and monitor progress. Your team should explicitly discuss:

- Core technical objectives and target distribution among members;
- Availability, individual schedules, and workload commitments;
- Relevant technical strengths (e.g. C debugging, fuzzing setup, Python scripting, technical documentation).

2.2 Canvas Registration

Register your team on the IFN657 Canvas portal (**People** → **Groups**) as early as possible, and no later than 30 September. It is strongly recommended that you form your own team. If your group is not registered by the deadline, the teaching team will assign you to an available team. You may also use the Canvas Discussion channel to connect with peers.

3 Target Software

You are provided with three vulnerable C firmware targets. To emulate real-world operational environments, source locations, counts, and vulnerability types are intentionally undisclosed. Each target exposes distinct parsing paradigms, ranging from shallow text processing to structured binary frames and stateful network packet streams.

Target component	Input paradigm	Architectural characteristics & attack surface
sentinel_telemetry	Text / key-value directives	Parses configuration lines, sensor IDs, calibration factors, and dynamic stream allocations. Diverse string operations and integer calculations.
sentinel_payload	Structured binary frame	Processes file headers (PAYLOAD_FRAME width height depth label), multi-dimensional matrix buffers, and cached observation frames.
sentinel_network	Binary packet stream	Processes streaming packets with magic header validation (0x12345678), packet types, length fields, and emergency failover queues.

3.1 Feasibility of Seed Identification & Starter Seeds

To make seed design accessible while encouraging systematic exploration, one valid starter seed is provided for each target in `targets/seeds/`:

- `seed_telemetry.conf`: Demonstrates valid key-value syntax, comments, and numeric directives.
- `seed_payload.bin`: Demonstrates a valid header line (`PAYLOAD_FRAME 8 8 1` `RADAR_SCAN_01\n`) followed by 64 bytes of sensor data.
- `seed_network.bin`: Demonstrates a valid binary packet with magic header `0x12345678`, Type 1, length 16, and nominal command payload bytes.

Students are required to expand these into 3 to 5 diverse, valid seed inputs per target. For binary targets (`sentinel_payload` and `sentinel_network`), students can construct structured seeds using Python scripts or binary utilities:

```
# Sample script to generate structured binary seeds
import struct

magic = 0x12345678
packet_type = 1          # Station status update type
payload_data = b"STATION_STATUS_NOMINAL_01"
packet_len = len(payload_data)

# Pack header: 4-byte uint32 magic, 2-byte uint16 type, 2-byte uint16 length
packet_header = struct.pack("<IHH", magic, packet_type, packet_len)

with open("seed_network_type1.bin", "wb") as f:
    f.write(packet_header + payload_data)
```

4 Fuzz Testing Project (30%)

You must compile the targets with appropriate compiler instrumentation and conduct your fuzzing campaigns in the same standard Linux environment used in tutorials and Assignment 1 (Ubuntu LTS with GCC, Clang, GDB, and 32-bit multilib libraries). Do not modify the original target C source code during compilation and testing.

The practical assessment is divided into five core technical tasks (Tasks 1-5), reflecting modern professional vulnerability research and remediation workflows.

4.1 Task 1: Setup (4%)

- Compile all three targets using AFL++ instrumentation.
- Construct a corpus of 3 to 5 valid, non-crashing seed inputs for **each** target.
- Document your seed generation methodology. Explain why your seeds are structurally valid and how they exercise different execution branches and parsing states.

4.2 Task 2: Fuzzing Execution (6%)

- Execute dedicated fuzzing campaigns on all three targets for a sufficient duration (longer campaigns yield higher path coverage).
- Implement a variety of fuzzing techniques to maximise campaign efficiency, such as multi-core parallel fuzzing, dictionaries, persistent in-process fuzzing or sanitiser-guided fuzzing.

- Document campaign metrics across all targets such as duration, total execution count, execution speed, and discovered total paths. Include clear screenshots of the AFL++ status console.

4.3 Task 3: Vulnerability Analysis (10%)

- Systematically triage all crashing inputs reported by AFL++.
- Report all distinct vulnerabilities discovered across the target programs, which cover diverse vulnerability classes (e.g. stack buffer overflow, heap buffer overflow, format string vulnerability, integer overflow/underflow, use-after-free, double-free, or uninitialised memory access).
- For each discovered vulnerability, provide a comprehensive technical analysis containing:
 - **Vulnerability identification:** Name, classification (CWE), target component, source file, and exact line number;
 - **Triggering input:** Exact reproducible input file that triggers the bug;
 - **Root cause analysis:** Detailed explanation of why the flaw occurs, including stack/heap state, pointer modifications, or arithmetic wraps;
 - **Debugger & sanitiser evidence:** Annotated GDB backtrace and sanitiser crash report verifying the exact point of failure;
 - **Exploitability evaluation:** Critical assessment of potential attacker impact (e.g. denial of service, arbitrary read/write, control flow hijacking).

4.4 Task 4: Code Remediation (6%)

- Provide precise, secure C code revisions for each identified vulnerability in the target programs.
- Explain the technical justification for each patch, showing how bounds enforcement, safe string handling, overflow checks, or memory lifecycle tracking neutralise the vulnerability without introducing regressions.
- Re-compile your patched targets and execute regression fuzzing campaigns using AFL++ and the previous crashing inputs. Demonstrate that the patched programs run cleanly without crashing while continuing to accept and process valid input streams.

4.5 Task 5: Proof-of-Concept Exploitation (4%)

- Select at least **one** critical vulnerability discovered during your triage (such as a stack buffer overflow, format string flaw, or heap corruption).
- Construct a fully functioning, actionable proof-of-concept exploit that achieves a tangible security outcome *beyond a simple program crash* (e.g. overriding an internal authorisation flag, forcing execution of an uncalled diagnostic function, leaking sensitive stack memory, or hijacking control flow).
- Provide your complete exploit script (e.g. Python), memory layout diagrams, resolved target addresses, and step-by-step shell execution commands demonstrating successful exploitation in standard bash.

5 Recording & Demonstration

To verify that your fuzzing campaigns and exploits are genuine and reproducible, your team must record and submit a terminal session (Group_XX.cast). In addition, your team will present your project during the Week 13 tutorial session.

5.1 Recording with `asciinema`

Recording Instructions and checklist:

1. **Start recording:** Run the following command before executing your demonstration:

```
> asciinema rec Group_XX.cast
```

2. **Team identification step:** At the very start of the session, display your team identification and environment details using the following commands (substituting your allocated group number and member details):

```
> echo "=== IFN657 ASSIGNMENT 2 TEAM IDENTIFICATION ==="
> echo "Group: Group_05 | Members: N1111111 (Alice), N2222222 (Bob), N3333333 (Carol), N4444444 (David)"
> whoami && hostname
> uname -a
> afl-fuzz -v && gcc --version
```

The team identification step is mandatory; omitting it will result in zero marks for the demonstration component.

3. **Fuzzing & exploit walkthrough (demonstrate all practical components):**
 - *Compilation:* Show compilation of all targets with AFL++ and AddressSanitizer.
 - *Fuzzing status:* Display at least two running AFL++ status screens showing active fuzzing campaigns, coverage metrics, and discovered crashes.
 - *Crash triage:* Reproduce at least two distinct crashes using GDB or sanitisers, demonstrating the memory access violation.
 - *Remediation verification:* Demonstrate running at least two patched targets against the original crashing inputs to confirm the bugs are resolved.
 - *Proof-of-concept exploit:* Execute your Python exploit script in bash, demonstrating the actionable exploit effect.
4. **Ending the recording:** Type `exit` or press `Ctrl+D` to conclude the recording session. Verify the recording locally:

```
> asciinema play Group_XX.cast
```

5.2 Team Presentation (During Week 13 Tutorial)

Before final grading, each team will present their project during their registered Week 13 tutorial class. Attendance is mandatory for all group members.

Presentation Format & Schedule:

- **Time allocation:** 8 minutes presentation followed by 2 minutes Q&A with the teaching team.

- **Format:** Live demonstration using terminal sessions, slides, or local environment.
- **Core presentation agenda:**
 1. *Fuzzing methodology:* Seed corpus design, dictionary generation, and parallel AFL++ deployment strategy.
 2. *Key vulnerability findings:* Walkthrough of the most interesting bug discovered, sanitiser triage logs, and memory corruption mechanisms.
 3. *Exploit demonstration:* Live execution of your actionable proof-of-concept exploit.
 4. *Remediation & reflection:* Sample patched source code, regression fuzzing results, and lessons learned.
 5. *Q&A session:* Individual technical questions to verify understanding and equitable contribution.

6 Submission & Deliverables

Only one team member must submit the required assessment deliverables via the IFN657 Canvas portal on behalf of the group before Friday 16 October (Week 12). The following two files must be submitted directly to Canvas:

6.1 Technical Report (Group_XX.pdf)

A comprehensive technical report compiled from the report template (maximum 20 pages, excluding appendix):

- It should include the completed Executive Summary task table, team workload distribution, seed designs, fuzzing statistics, detailed vulnerability analyses, remediation code diffs, exploit documentation, and individual workload breakdowns.
- The standardised Markdown report template (`as2_report_template.md`) is provided for this assessment and available from the Assignment's Canvas page. You must complete the sections and then export it to `Group_XX.pdf`.

6.2 Project Archive Package (Group_XX.zip)

A unified archive containing all practical assessment files:

- `seeds/`: Seed inputs used for all three targets;
- `crashes/`: Unique crashing inputs reproducing each reported bug;
- `exploits/`: Python exploit script, payload files, and execution instructions;
- `remediated_src/`: Securely patched C source code files (`sentinel_telemetry.c`, `sentinel_payload.c`, `sentinel_network.c`);
- `README.md`: Manifest file documenting the purpose, syntax, and execution instructions for every file in the ZIP archive;
- `Group_XX.cast`: The validated asciinema recording file.

7 Marking Criteria (30 Points Total)

Your group submission will be evaluated based on the depth of technical analysis, fuzzing rigour, exploit execution, code fixes, and demonstration clarity.

To emulate authentic software security audits and industry fuzz testing, the total number, locations, and types of vulnerabilities across the three firmware targets are intentionally

not disclosed to students. Teams should conduct thorough, multi-faceted fuzzing campaigns and strive to discover as many distinct vulnerabilities as possible across all three components.

Important grading rule: Marks awarded for vulnerability analysis and code remediation are directly capped by the number of valid, distinct vulnerabilities discovered and verified by your team. Reporting fewer vulnerabilities will proportionally reduce the maximum achievable marks for those components.

Task	Mark	Description
Task 0: Claims, Demo & Teamwork	N/A (Gatekeeper)	A claim table accurately stating each task completion and team workload distribution. Validated asciinema recording with mandatory identification. Project presentation delivery in the Week 13 tutorial.
Task 1: Setup	4 pts	Correct compilation with AFL++/sanitisers, diverse valid seed sets for all 3 targets, and technical justification of seed structure.
Task 2: Fuzzing Execution	6 pts	Effective AFL++ fuzzing campaigns with core optimisation techniques like parallel execution, effective dictionary utilisation, persistent/in-process modes, and thorough coverage/metric analysis.
Task 3: Vulnerability Analysis	10 pts	Comprehensive triage of crashes using GDB and sanitisers. Thorough technical report on the distinct vulnerabilities found across targets (root causes, triggering inputs, memory corruption mechanics, exploitability).
Task 4: Code Remediation	6 pts	Sound code-level C fixes for all identified bugs, technical justification, and regression fuzzing proof showing zero crashes while preserving functionality.
Task 5: Proof-of-Concept Exploitation	4 pts	Working proof-of-concept exploit demonstrating actionable impact beyond a crash, reproducible script, memory analysis, and bash execution evidence.